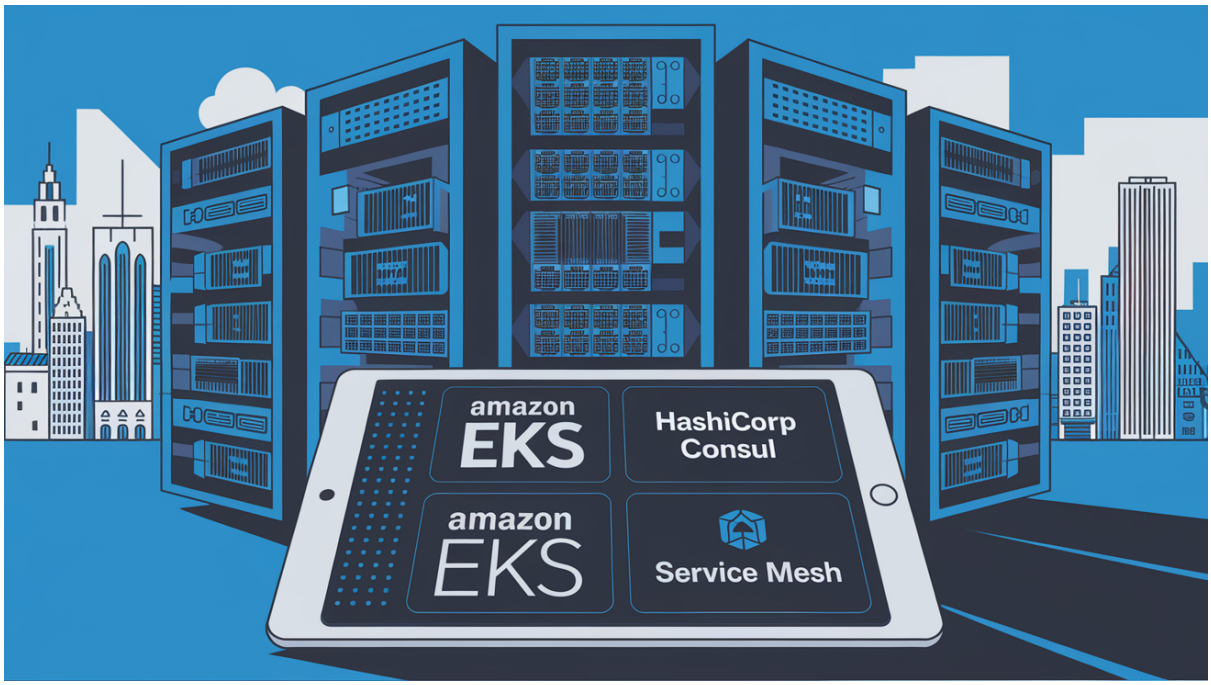
Building Secure and Scalable Microservices with Amazon EKS and HashiCorp Consul Service Mesh



By
Mahendran Selvakumar
https://devopstronaut.com/

https://www.linkedin.com/in/mahendran-selvakumar/

What is a Service Mesh?

A **service mesh** is a tool that helps services in a microservices architecture communicate with each other securely and efficiently. It manages tasks like routing traffic, encrypting data between services, and monitoring how they interact—all without needing to change the application code.

Key Benefits:

- **Secure Communication**: Encrypts data between services using mTLS.
- **Traffic Control**: Handles routing, load balancing, and retries to improve performance.
- **Better Visibility**: Tracks and monitors service interactions for easier troubleshooting.
- **Reliability**: Adds features like failovers and rate limiting to ensure smooth operations.

What is HashiCorp Consul?

HashiCorp Consul is a **service networking solution** designed to simplify and secure communication between services in modern, distributed systems. It provides features like **service discovery, service mesh, and configuration management**, enabling microservices to connect and interact seamlessly.

Key Features:

1. **Service Discovery**: Automatically registers and discovers services in your network, eliminating the need for manual service management.
2. **Service Mesh**: Enables secure, encrypted service-to-service communication with traffic routing, load balancing, and observability.
3. **Intentions**: Allows fine-grained access control by defining rules (e.g., allow or deny) for communication between services.
4. **Multi-Cloud Support**: Works across multiple cloud providers and hybrid environments, making it ideal for scalable, global workloads.
5. **Health Monitoring**: Continuously checks the health of services and routes traffic only to healthy instances.
6. **Dynamic Configuration**: Stores and distributes configuration changes across your services in real-time.

Use Case: Secure and Scalable Service-to-Service Communication

This setup is ideal for organizations running **microservices architectures** on Amazon EKS. By deploying HashiCorp Consul with a service mesh, you can:

1. **Secure Service Communication**: Use Consul intentions to enforce granular traffic policies, allowing or denying service interactions, ensuring security by default.
2. **Simplify Multi-Service Deployments**: Consul's service discovery and built-in service mesh features make it easy to deploy and manage multiple interconnected services, such as frontend and backend applications.
3. **Enable Observability**: Consul integrates with monitoring tools, providing visibility into service dependencies and communication flows.

4. **Scalable Infrastructure**: The EKS cluster with managed node groups and auto-scaling capabilities ensures the infrastructure scales dynamically with workload demands.
5. **AWS Integration**: Leverages AWS IAM and storage classes (e.g., EBS) for secure and reliable resource management, making it ideal for cloud-native applications.

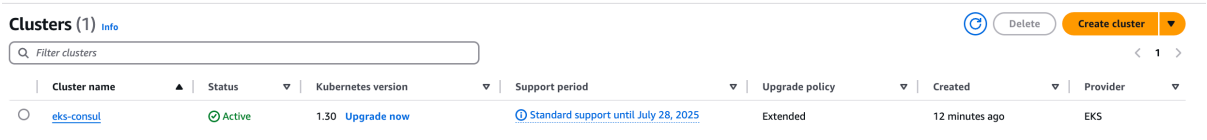
This solution is particularly useful for **financial services, e-commerce platforms, and SaaS providers**, where secure, reliable, and scalable service interactions are critical.

Step 1: Create the EKS Cluster Without Any Node Groups

Create an EKS cluster without a node group using the eksctl command **(eksctl create cluster --name=eks-consul --region=eu-west-1 --without-nodegroup)**. By default, eksctl creates a node group with m5.large instances, so we used the — without-nodegroup option to skip creating a default node group

```
mahendranselvakumar@Mahendrans-MacBook-Pro ~ % eksctl create cluster --name=eks-consul --region=eu-west-1 --without-nodegroup
2024-12-24 00:23:05 [ℹ]  eksctl version 0.196.0
2024-12-24 00:23:05 [ℹ]  using region eu-west-1
2024-12-24 00:23:05 [ℹ]  setting availability zones to [eu-west-1b eu-west-1c eu-west-1a]
2024-12-24 00:23:05 [ℹ]  subnets for eu-west-1b - public:192.168.0.0/19 private:192.168.96.0/19
2024-12-24 00:23:05 [ℹ]  subnets for eu-west-1c - public:192.168.32.0/19 private:192.168.128.0/19
2024-12-24 00:23:05 [ℹ]  subnets for eu-west-1a - public:192.168.64.0/19 private:192.168.160.0/19
2024-12-24 00:23:05 [ℹ]  using Kubernetes version 1.30
2024-12-24 00:23:05 [ℹ]  creating EKS cluster "eks-consul" in "eu-west-1" region with
2024-12-24 00:23:05 [ℹ]  if you encounter any issues, check CloudFormation console or try 'eksctl utils describe-stacks --region=eu-west-1 --cluster=eks-consul'
2024-12-24 00:23:05 [ℹ]  Kubernetes API endpoint access will use default of {publicAccess=true, privateAccess=false} for cluster "eks-consul" in "eu-west-1"
2024-12-24 00:23:05 [ℹ]  CloudWatch logging will not be enabled for cluster "eks-consul" in "eu-west-1"
2024-12-24 00:23:05 [ℹ]  you can enable it with 'eksctl utils update-cluster-logging --enable-types={SPECIFY-YOUR-LOG-TYPES-HERE (e.g. all)} --region=eu-west-1 --cluster=eks-consul'
2024-12-24 00:23:05 [ℹ]  default addons vpc-cni, kube-proxy, coredns were not specified, will install them as EKS addons
2024-12-24 00:23:05 [ℹ]
2 sequential tasks: { create cluster control plane "eks-consul",
    2 sequential sub-tasks: {
        1 task: { create addons },
        wait for control plane to become ready,
    }
}
2024-12-24 00:23:05 [ℹ]  building cluster stack "eksctl-eks-consul-cluster"
2024-12-24 00:23:06 [ℹ]  deploying stack "eksctl-eks-consul-cluster"
2024-12-24 00:23:36 [ℹ]  waiting for CloudFormation stack "eksctl-eks-consul-cluster"
2024-12-24 00:24:06 [ℹ]  waiting for CloudFormation stack "eksctl-eks-consul-cluster"
2024-12-24 00:25:06 [ℹ]  waiting for CloudFormation stack "eksctl-eks-consul-cluster"
2024-12-24 00:26:06 [ℹ]  waiting for CloudFormation stack "eksctl-eks-consul-cluster"
2024-12-24 00:27:06 [ℹ]  waiting for CloudFormation stack "eksctl-eks-consul-cluster"
2024-12-24 00:28:06 [ℹ]  waiting for CloudFormation stack "eksctl-eks-consul-cluster"
2024-12-24 00:29:07 [ℹ]  waiting for CloudFormation stack "eksctl-eks-consul-cluster"
2024-12-24 00:30:07 [ℹ]  waiting for CloudFormation stack "eksctl-eks-consul-cluster"
2024-12-24 00:31:07 [ℹ]  waiting for CloudFormation stack "eksctl-eks-consul-cluster"
2024-12-24 00:32:44 [ℹ]  waiting for CloudFormation stack "eksctl-eks-consul-cluster"
2024-12-24 00:32:46 [!]  recommended policies were found for "vpc-cni" addon, but since OIDC is disabled on the cluster, eksctl cannot configure the requested permissions; the recommended
 way to provide IAM permissions for "vpc-cni" addon is via pod identity associations; after addon creation is completed, add all recommended policies to the config file, under `addon.PodI
dentityAssociations`, and run `eksctl update addon`
2024-12-24 00:32:46 [ℹ]  creating addon
2024-12-24 00:32:46 [ℹ]  successfully created addon
2024-12-24 00:32:46 [ℹ]  creating addon
2024-12-24 00:32:47 [ℹ]  successfully created addon
2024-12-24 00:32:47 [ℹ]  creating addon
2024-12-24 00:32:47 [ℹ]  successfully created addon
2024-12-24 00:34:48 [ℹ]  waiting for the control plane to become ready
2024-12-24 00:34:48 [✔]  saved kubeconfig as "/Users/mahendranselvakumar/.kube/config"
2024-12-24 00:34:48 [ℹ]  no tasks
2024-12-24 00:34:48 [✔]  all EKS cluster resources for "eks-consul" have been created
2024-12-24 00:34:48 [ℹ]  kubectl command should work with "/Users/mahendranselvakumar/.kube/config", try 'kubectl get nodes'
2024-12-24 00:34:48 [✔]  EKS cluster "eks-consul" in "eu-west-1" region is ready
mahendranselvakumar@Mahendrans-MacBook-Pro ~ %
```

Go to the EKS console to verify that the cluster was successfully created using eksctl



Step 2: Create a Managed Node Group

Add a node group using the following separate eksctl command: **eksctl create nodegroup --name consul-ng --cluster eks-consul --region eu-west-1 --nodes 2 --nodes-min 1 --nodes-max 3 --node-type t3.medium**

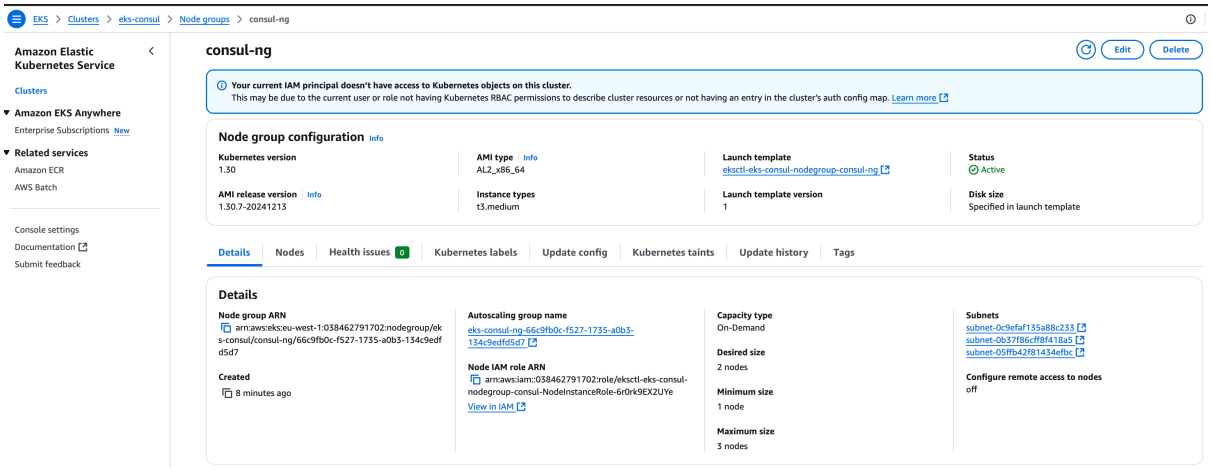
```
mahendranselvakumar@Mahendrans-MacBook-Pro ~ % eksctl create nodegroup --name consul-ng --cluster eks-consul --region eu-west-1 --nodes 2 --nodes-min 1 --nodes-max 3 --node-type t3.medium

2024-12-24 00:38:09 [ℹ]  will use version 1.30 for new nodegroup(s) based on control plane version
2024-12-24 00:38:11 [ℹ]  nodegroup "consul-ng" will use "" [AmazonLinux2/1.30]
2024-12-24 00:38:12 [ℹ]  1 nodegroup (consul-ng) was included (based on the include/exclude rules)
2024-12-24 00:38:12 [ℹ]  will create a CloudFormation stack for each of 1 managed nodegroups in cluster "eks-consul"
2024-12-24 00:38:12 [ℹ]
2 sequential tasks: { fix cluster compatibility, 1 task: { 1 task: { create managed nodegroup "consul-ng" } }
}
2024-12-24 00:38:12 [ℹ]  checking cluster stack for missing resources
2024-12-24 00:38:12 [ℹ]  cluster stack has all required resources
2024-12-24 00:38:12 [ℹ]  building managed nodegroup stack "eksctl-eks-consul-nodegroup-consul-ng"
2024-12-24 00:38:12 [ℹ]  deploying stack "eksctl-eks-consul-nodegroup-consul-ng"
2024-12-24 00:38:12 [ℹ]  waiting for CloudFormation stack "eksctl-eks-consul-nodegroup-consul-ng"
2024-12-24 00:38:42 [ℹ]  waiting for CloudFormation stack "eksctl-eks-consul-nodegroup-consul-ng"
2024-12-24 00:39:16 [ℹ]  waiting for CloudFormation stack "eksctl-eks-consul-nodegroup-consul-ng"
2024-12-24 00:39:58 [ℹ]  waiting for CloudFormation stack "eksctl-eks-consul-nodegroup-consul-ng"
2024-12-24 00:40:53 [ℹ]  waiting for CloudFormation stack "eksctl-eks-consul-nodegroup-consul-ng"
2024-12-24 00:40:53 [ℹ]  no tasks
2024-12-24 00:40:53 [✔]  created 0 nodegroup(s) in cluster "eks-consul"
2024-12-24 00:40:53 [ℹ]  nodegroup "consul-ng" has 2 node(s)
2024-12-24 00:40:53 [ℹ]  node "ip-192-168-0-106.eu-west-1.compute.internal" is ready
2024-12-24 00:40:53 [ℹ]  node "ip-192-168-55-167.eu-west-1.compute.internal" is ready
2024-12-24 00:40:53 [ℹ]  waiting for at least 1 node(s) to become ready in "consul-ng"
2024-12-24 00:40:53 [ℹ]  nodegroup "consul-ng" has 2 node(s)
2024-12-24 00:40:53 [ℹ]  node "ip-192-168-0-106.eu-west-1.compute.internal" is ready
2024-12-24 00:40:53 [ℹ]  node "ip-192-168-55-167.eu-west-1.compute.internal" is ready
2024-12-24 00:40:53 [✔]  created 1 managed nodegroup(s) in cluster "eks-consul"
2024-12-24 00:40:53 [ℹ]  checking security group configuration for all nodegroups
2024-12-24 00:40:53 [ℹ]  all nodegroups have up-to-date cloudformation templates
mahendranselvakumar@Mahendrans-MacBook-Pro ~ %
```

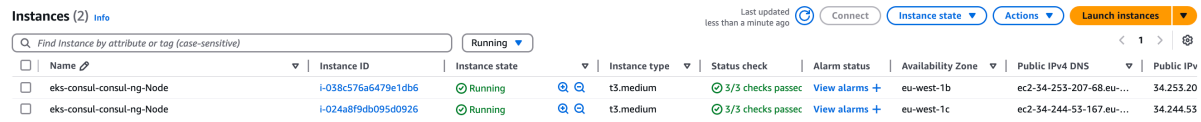
Explanation of the flags:

- — cluster: Specifies the name of the existing EKS cluster to which the node group will be added.
- — name: Names the node group for easy identification.
- — region: Specifies the AWS region.
- — nodes: Sets the initial desired number of nodes (in this case, 2).
- — nodes-min and — nodes-max: Define the minimum and maximum number of nodes for auto-scaling.
- — node-type: The EC2 instance type for the nodes (e.g., m5.large)

If you check in the console, you'll see that the node group has been created



In the instances section of the EC2 console, you can view the Instances created for the node group



Step 3: Configure Context for EKS Cluster

Set the Kubernetes context for the EKS cluster using the following command: **aws eks --region eu-west-1 update-kubeconfig --name eks-consul**

```
mahendranselvakumar@Mahendrans-MacBook-Pro ~ % aws eks --region eu-west-1 update-kubeconfig --name eks-consul
Added new context arn:aws:eks:eu-west-1:038462791702:cluster/eks-consul to /Users/mahendranselvakumar/.kube/config
mahendranselvakumar@Mahendrans-MacBook-Pro ~ %
```

Use **kubectl get ns** to view namespaces and **kubectl get nodes** to check the status of the nodes in the cluster, verifying that the setup is complete

```
mahendranselvakumar@Mahendrans-MacBook-Pro ~ % kubectl get ns
NAME              STATUS   AGE
default           Active   22m
kube-node-lease   Active   22m
kube-public       Active   22m
kube-system       Active   22m
mahendranselvakumar@Mahendrans-MacBook-Pro ~ % kubectl get nodes
NAME                                           STATUS   ROLES    AGE   VERSION
ip-192-168-0-106.eu-west-1.compute.internal    Ready    <none>   12m   v1.30.7-eks-59bf375
ip-192-168-55-167.eu-west-1.compute.internal   Ready    <none>   12m   v1.30.7-eks-59bf375
mahendranselvakumar@Mahendrans-MacBook-Pro ~ %
```

Step 4: Associate an IAM OIDC provider with your EKS cluster

Run this command to create an OIDC identity provider for your EKS cluster **(eksctl utils associate-iam-oidc-provider --region=eu-west-1 --cluster=eks-consul --approve)**. This step is necessary to enable IAM roles for service accounts, allowing Kubernetes workloads to securely interact with AWS services

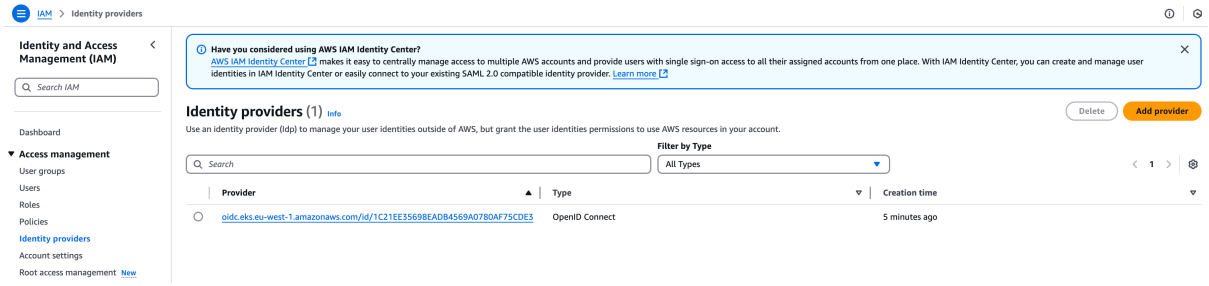
```
mahendranselvakumar@Mahendrans-MacBook-Pro ~ % eksctl utils associate-iam-oidc-provider --region=eu-west-1 --cluster=eks-consul --approve
2024-12-24 00:59:43 [ℹ]  will create IAM Open ID Connect provider for cluster "eks-consul" in "eu-west-1"
2024-12-24 00:59:44 [✔]  created IAM Open ID Connect provider for cluster "eks-consul" in "eu-west-1"
mahendranselvakumar@Mahendrans-MacBook-Pro ~ %
```

eksctl utils associate-iam-oidc-provider: This command creates an OIDC identity provider for your EKS cluster, which is necessary to allow Kubernetes service accounts to assume IAM roles.

--region=eu-west-1: Specifies the AWS region of your EKS cluster.

--cluster=devopstronaut-cluster: Specifies the name of your EKS cluster.

--approve: Automatically approves the operation without interactive confirmation.
Once the command completes, you can verify the OIDC provider association by checking under **IAM** > **Identity Providers** in the AWS Management Console



Step 5: Create a Namespace for the Application

Create a namespace using this command **kubectl create namespace consul**

```
mahendranselvakumar@Mahendrans-MacBook-Pro ~ % kubectl create namespace consul
namespace/consul created
mahendranselvakumar@Mahendrans-MacBook-Pro ~ %
```

Verify the namespace using this command **kubectl get ns**

```
mahendranselvakumar@Mahendrans-MacBook-Pro ~ % kubectl get ns
NAME              STATUS   AGE
consul            Active   67s
default           Active   44m
kube-node-lease   Active   44m
kube-public       Active   44m
kube-system       Active   44m
mahendranselvakumar@Mahendrans-MacBook-Pro ~ %
```

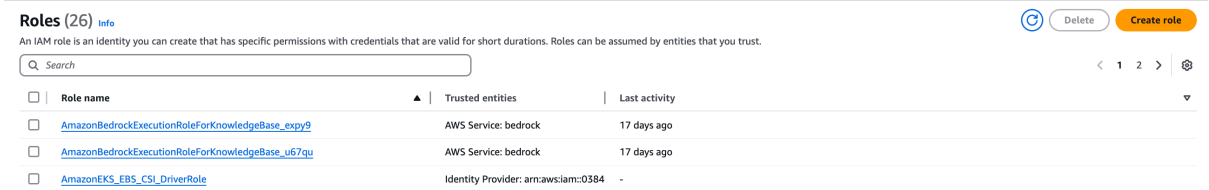
Step 6: Install AWS EBS CSI Driver

Run the following command to create an IAM service account with the policies needed to use the Amazon EBS CSI Driver

eksctl create iamserviceaccount --name ebs-csi-controller-sa --namespace kube-system --cluster eks-consul --attach-policy-arn arn:aws:iam::aws:policy/service-role/AmazonEBSCSIDriverPolicy --approve --role-only --role-name AmazonEKS_EBS_CSI_DriverRole

```
mahendranselvakumar@Mahendrans-MacBook-Pro ~ % eksctl create iamserviceaccount --name ebs-csi-controller-sa --namespace kube-system --cluster eks-consul --attach-policy-arn arn:aws:iam::aws:policy/service-role/AmazonEBSCSIDriverPolicy --approve --role-only --role-name AmazonEKS_EBS_CSI_DriverRole
2024-12-24 01:39:13 [ℹ]  1 iamserviceaccount (kube-system/ebs-csi-controller-sa) was included (based on the include/exclude rules)
2024-12-24 01:39:13 [!]  serviceaccounts in Kubernetes will not be created or modified, since the option --role-only is used
2024-12-24 01:39:13 [ℹ]  1 task: { create IAM role for serviceaccount "kube-system/ebs-csi-controller-sa" }
2024-12-24 01:39:13 [ℹ]  building iamserviceaccount stack "eksctl-eks-consul-addon-iamserviceaccount-kube-system-ebs-csi-controller-sa"
2024-12-24 01:39:14 [ℹ]  deploying stack "eksctl-eks-consul-addon-iamserviceaccount-kube-system-ebs-csi-controller-sa"
2024-12-24 01:39:14 [ℹ]  waiting for CloudFormation stack "eksctl-eks-consul-addon-iamserviceaccount-kube-system-ebs-csi-controller-sa"
2024-12-24 01:39:44 [ℹ]  waiting for CloudFormation stack "eksctl-eks-consul-addon-iamserviceaccount-kube-system-ebs-csi-controller-sa"
mahendranselvakumar@Mahendrans-MacBook-Pro ~ %
```

An IAM role named AmazonEKS_EBS_CSI_DriverRole has been created



Install the Amazon EBS CSI add-on for EKS using the AmazonEKS_EBS_CSI_DriverRole by issuing this command

eksctl create addon --name aws-ebs-csi-driver --cluster eks-consul --service-account-role-arn arn:aws:iam::038462791702:role/AmazonEKS_EBS_CSI_DriverRole –force

```
mahendranselvakumar@Mahendrans-MacBook-Pro ~ % eksctl create addon --name aws-ebs-csi-driver --cluster eks-consul --service-account-role-arn arn:aws:iam::038462791702:role/AmazonEKS_EBS_CSI_DriverRole –force

2024-12-24 01:48:21 [ℹ]  Kubernetes version "1.30" in use by cluster "eks-consul"
2024-12-24 01:48:21 [ℹ]  IRSA is set for "aws-ebs-csi-driver" addon; will use this to configure IAM permissions
2024-12-24 01:48:21 [!]  the recommended way to provide IAM permissions for "aws-ebs-csi-driver" addon is via pod identity associations; after addon creation is completed, run `eksctl utils migrate-to-pod-identity`
2024-12-24 01:48:21 [ℹ]  using provided ServiceAccountRoleARN "arn:aws:iam::038462791702:role/AmazonEKS_EBS_CSI_DriverRole"
2024-12-24 01:48:21 [ℹ]  creating addon
mahendranselvakumar@Mahendrans-MacBook-Pro ~ %
```

Now, check the EKS cluster add-ons to confirm that the Amazon EBS CSI Driver is installed using this command **(eksctl get addons --cluster eks-consul)**

```
mahendranselvakumar@Mahendrans-MacBook-Pro ~ % eksctl get addons --cluster eks-consul
2024-12-24 01:50:20 [ℹ]  Kubernetes version "1.30" in use by cluster "eks-consul"
2024-12-24 01:50:20 [ℹ]  getting all addons
2024-12-24 01:50:22 [ℹ]  to see issues for an addon run `eksctl get addon --name <addon-name> --cluster <cluster-name>`
NAME                    VERSION                 STATUS  ISSUES  IAMROLE                                                         UPDATE AVAILABLE                                                C
ONFIGURATION VALUES     POD IDENTITY ASSOCIATION ROLES
aws-ebs-csi-driver      v1.37.0-eksbuild.1      ACTIVE  0       arn:aws:iam::038462791702:role/AmazonEKS_EBS_CSI_DriverRole
coredns                 v1.11.1-eksbuild.8      ACTIVE  0                                                                       v1.11.3-eksbuild.2,v1.11.3-eksbuild.1,v1.11.1-eksbuild.13,v
1.11.1-eksbuild.11,v1.11.1-eksbuild.9
kube-proxy              v1.30.6-eksbuild.3      ACTIVE  0                                                                       v1.30.7-eksbuild.2
vpc-cni                 v1.19.0-eksbuild.1      ACTIVE  0
mahendranselvakumar@Mahendrans-MacBook-Pro ~ %
```

Step 7: Deploy HashiCorp Consul on EKS

Add the Consul Helm Chart Repository using this commands **helm repo add hashicorp** https://helm.releases.hashicorp.com and **helm repo update**

```
mahendranselvakumar@Mahendrans-MacBook-Pro ~ % helm repo add hashicorp https://helm.releases.hashicorp.com
"hashicorp" has been added to your repositories
mahendranselvakumar@Mahendrans-MacBook-Pro ~ % helm repo update
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "hashicorp" chart repository
...Successfully got an update from the "aws-observability" chart repository
...Successfully got an update from the "eks" chart repository
...Successfully got an update from the "kubecost" chart repository
...Successfully got an update from the "mybitnami" chart repository
Update Complete. ⎈Happy Helming!⎈
mahendranselvakumar@Mahendrans-MacBook-Pro ~ %
```

Deploy Consul with the service mesh enabled using this command **helm install consul hashicorp/consul --namespace consul --set global.name=consul --set global.datacenter=eks-dc --set connectInject.enabled=true**

```
mahendranselvakumar@Mahendrans-MacBook-Pro ~ % helm install consul hashicorp/consul \
    --namespace consul \
    --set global.name=consul \
    --set global.datacenter=eks-dc \
    --set connectInject.enabled=true

NAME: consul
LAST DEPLOYED: Tue Dec 24 01:19:30 2024
NAMESPACE: consul
STATUS: deployed
REVISION: 1
NOTES:
Thank you for installing HashiCorp Consul!

Your release is named consul.

To learn more about the release, run:

  $ helm status consul --namespace consul
  $ helm get all consul --namespace consul

Consul on Kubernetes Documentation:
https://www.consul.io/docs/platform/k8s

Consul on Kubernetes CLI Reference:
https://www.consul.io/docs/k8s/k8s-cli
mahendranselvakumar@Mahendrans-MacBook-Pro ~ %
```

Ensure Consul pods are running using this command **kubectl get pods -n consul**

```
mahendranselvakumar@Mahendrans-MacBook-Pro ~ % kubectl get pods -n consul
NAME                                           READY   STATUS             RESTARTS        AGE
consul-connect-injector-5c486f4488-qthp5       0/1     CrashLoopBackOff   11 (5m6s ago)   31m
consul-server-0                                0/1     Pending            0               31m
consul-webhook-cert-manager-5c78f7fc77-4mqk7   1/1     Running            0               31m
mahendranselvakumar@Mahendrans-MacBook-Pro ~ %
```

If your pods are showing a “Pending” status, use the describe command to check for details

```
mahendranselvakumar@Mahendrans-MacBook-Pro ~ % kubectl describe pod consul-server-0 -n consul

Name:             consul-server-0
Namespace:        consul
Priority:         0
Service Account:  consul-server
Node:             <none>
Labels:           app=consul
                  apps.kubernetes.io/pod-index=0
                  chart=consul-helm
                  component=server
                  controller-revision-hash=consul-server-85d9854447
                  hasDNS=true
                  release=consul
                  statefulset.kubernetes.io/pod-name=consul-server-0
Annotations:      consul.hashicorp.com/config-checksum: eabe4e9b616c4dc11a6be0706a8d65ae5b05e6616d8476de5bb485d2b50ca347
                  consul.hashicorp.com/connect-inject: false
                  consul.hashicorp.com/mesh-inject: false
Status:           Pending
```

Here it shows the pod has an “unbound PersistentVolumeClaim” error

```
                         node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type     Reason            Age              From              Message
  ----     ------            ----             ----              -------
  Warning  FailedScheduling  99s (x9 over 36m)  default-scheduler  0/2 nodes are available: pod has unbound immediate PersistentVolumeClaims. preemption: 0/2 nodes are available: 2 Preemption is not helpful for scheduling.
mahendranselvakumar@Mahendrans-MacBook-Pro ~ %
```

To resolve the above error, run the **kubectl get pvc --namespace consul** command to check the status of the PersistentVolumeClaims (PVCs). This will help identify any PVCs that are unbound and causing the issue

```
mahendranselvakumar@Mahendrans-MacBook-Pro ~ % kubectl get pvc --namespace consul
NAME                          STATUS    VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
data-consul-consul-server-0   Pending                                                      <unset>                 38m
mahendranselvakumar@Mahendrans-MacBook-Pro ~ %
```

Use the command **kubectl describe pvc data-consul-consul-server-0 --namespace consul** to get a detailed error message for the PersistentVolumeClaim. This will provide insights into why the PVC is unbound and help you troubleshoot the issue

```
mahendranselvakumar@Mahendrans-MacBook-Pro ~ % kubectl describe pvc data-consul-consul-server-0 --namespace consul
Name:          data-consul-consul-server-0
Namespace:     consul
StorageClass:
Status:        Pending
Volume:
Labels:        app=consul
               chart=consul-helm
               component=server
               hasDNS=true
               release=consul
Annotations:   <none>
Finalizers:    [kubernetes.io/pvc-protection]
Capacity:
Access Modes:
VolumeMode:    Filesystem
Used By:       consul-server-0
Events:
  Type    Reason         Age                From                         Message
  ----    ------         ----               ----                         -------
  Normal  FailedBinding  22s (x162 over 40m)  persistentvolume-controller  no persistent volumes available for this claim and no storage class is set
mahendranselvakumar@Mahendrans-MacBook-Pro ~ %
```

If you see the error message **"no persistent volumes available for this claim and no storage class"** it indicates that the PVC is missing a storage class. To resolve this, edit the PVC (**kubectl edit pvc data-consul-consul-server-0 --namespace consul)** to include **storageClassName: gp2** under the spec section, **as gp2 is an AWS storage class.**

This will direct Kubernetes to use the gp2 storage class, enabling the PVC to bind to a suitable PersistentVolume

```
# Please edit the object below. Lines beginning with a '#' will be ignored,
# and an empty file will abort the edit. If an error occurs while saving this file will be
# reopened with the relevant failures.
#
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  creationTimestamp: "2024-12-24T01:19:35Z"
  finalizers:
  - kubernetes.io/pvc-protection
  labels:
    app: consul
    chart: consul-helm
    component: server
    hasDNS: "true"
    release: consul
  name: data-consul-consul-server-0
  namespace: consul
  resourceVersion: "8808"
  uid: fa548674-a974-4b23-bd8d-ff3f6cb7d28a
spec:
  accessModes:
  - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
  volumeMode: Filesystem
  storageClassName: gp2
status:
  phase: Pending
```

After adding **storageClassName: gp2** to the PVC configuration, save and close the file. This update will prompt Kubernetes to attempt binding the PersistentVolumeClaim to an available volume using the specified storage class.Run **kubectl get pvc --namespace consul** command to get the pvc detail

```
mahendranselvakumar@Mahendrans-MacBook-Pro ~ % kubectl get pvc --namespace consul
NAME                        STATUS   VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
data-consul-consul-server-0   Bound    pvc-fa548674-a974-4b23-bd8d-ff3f6cb7d28a   10Gi       RWO            gp2            <unset>                 44m
mahendranselvakumar@Mahendrans-MacBook-Pro ~ %
```

Now the pods should be in a running state. You can verify this by using the command: **kubectl get pods -n consul**

```
mahendranselvakumar@Mahendrans-MacBook-Pro ~ % kubectl get pods -n consul
NAME                                          READY   STATUS    RESTARTS         AGE
consul-connect-injector-5c486f4488-qthp5      0/1     Running   16 (5m31s ago)   46m
consul-server-0                               1/1     Running   0                46m
consul-webhook-cert-manager-5c78f7fc77-4mqk7  1/1     Running   0                46m
mahendranselvakumar@Mahendrans-MacBook-Pro ~ %
```

This will display the status of the pods in the consul namespace, confirming that they are now running successfully.

Step 8: Deploy Example Applications

We'll deploy two simple applications **frontend** and **backend** Both will have Consul sidecars for service-to-service communication.

Deploy the Backend Service:

Create a **backend-deployment.yaml** file with the following code

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  namespace: consul
  annotations:
    "consul.hashicorp.com/connect-inject": "true"
spec:
  replicas: 1
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
        - name: backend
          image: hashicorp/http-echo
          args:
            - "-text=Hello from Backend"
          ports:
            - containerPort: 8080
```

Apply the deployment using this command **kubectl apply -f backend-deployment.yaml**

```
mahendranselvakumar@Mahendrans-MacBook-Pro ~ % kubectl apply -f backend-deployment.yaml

deployment.apps/backend created
mahendranselvakumar@Mahendrans-MacBook-Pro ~ %
```

To Expose the backend service,create a backend-service.yaml file with the following code

Apply the service using this command **kubectl apply -f backend-service.yaml**

```
mahendranselvakumar@Mahendrans-MacBook-Pro ~ % kubectl apply -f backend-service.yaml

service/backend created
mahendranselvakumar@Mahendrans-MacBook-Pro ~ %
```

Deploy the Frontend Service:

Create a **frontend-deployment.yaml** file with the following code

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
  namespace: consul
  annotations:
    "consul.hashicorp.com/connect-inject": "true"
spec:
  replicas: 1
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:
        app: frontend
    spec:
      containers:
        - name: frontend
          image: hashicorp/http-echo
          args:
            - "-text=Hello from Frontend"
          ports:
            - containerPort: 8080
```

Apply the deployment using this command **kubectl apply -f frontend-deployment.yaml**

```
mahendranselvakumar@Mahendrans-MacBook-Pro ~ % kubectl apply -f frontend-deployment.yaml

deployment.apps/frontend created
mahendranselvakumar@Mahendrans-MacBook-Pro ~ %
```

To expose the backend service, create a frontend-service.yaml file with the following code

```yaml
apiVersion: v1
kind: Service
metadata:
  name: frontend
  namespace: consul
spec:
  ports:
    - port: 80
      targetPort: 8080
  selector:
    app: frontend
```

Apply the service using this command **kubectl apply -f frontend-service.yaml**

```
mahendranselvakumar@Mahendrans-MacBook-Pro ~ % kubectl apply -f frontend-service.yaml

service/frontend created
mahendranselvakumar@Mahendrans-MacBook-Pro ~ %
```

Now both the backend and frontend applications have been deployed, and their services have also been exposed.

If you want to deploy applications in a **different namespace** while using HashiCorp Consul service mesh, you must enable **namespace access** in consul**. --set global.enableNamespaces=true** in the Consul Helm chart enables Consul to support and manage services across multiple Kubernetes namespaces. Here's what this configuration does:

Step 9: Configure Consul Intentions

To configure Consul intensions,we need to access the Consul Server Pod.Run this command to access the pod **kubectl exec -it consul-server-0 -n consul --sh**

```
mahendranselvakumar@Mahendrans-MacBook-Pro ~ % kubectl exec -it consul-server-0 -n consul -- sh

Defaulted container "consul" out of: consul, locality-init (init)
/ $
```

Run consul intension create -deny '*' '*' command to deny All Traffic by Default

```
Defaulted container "consul" out of: consul, locality-init (init)
/ $ consul intention create -deny '*'
Error: Must specify two arguments: source and destination
/ $ consul intention create -deny '*' '*'
Created: * => * (deny)
/ $
```

This sets a default policy that denies all traffic between any source and destination services
Allow Frontend to Communicate with Backend

Run **consul intention create -allow frontend backend** command to allow frontend to communicate with backend

```
/ $ consul intention create -allow frontend backend
Created: frontend => backend (allow)
/ $
```

Verify Intentions using **consul intentions list** command

```
/ $ consul intention list
ID                                    Source    Action  Destination  Precedence
2f934c0b-3626-6faf-5b74-c048148afb1d  frontend  allow   backend      9
ecac4aa0-87e5-6f16-69a3-c09feab3cf64  *         deny    *            5
/ $
```

- A `deny` rule blocking all traffic.
- An `allow` rule permitting `frontend` to communicate with `backend`

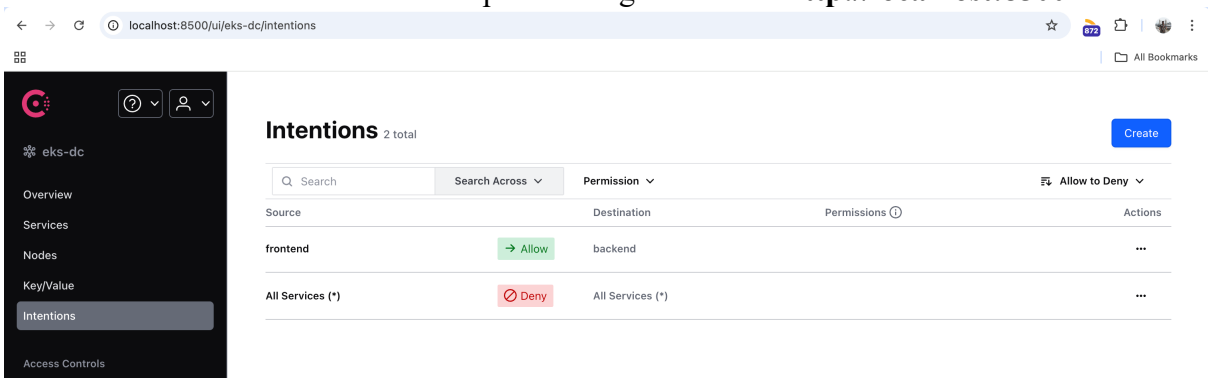
Step 10: Verify Intension in Consul Portal

`Run this command kubectl port-forward svc/consul-server 8500:8500 -n consul` to expose consul-server to local

```
mahendranselvakumar@Mahendrans-MacBook-Pro ~ % kubectl port-forward svc/consul-server 8500:8500 -n consul

Forwarding from 127.0.0.1:8500 -> 8500
Forwarding from [::1]:8500 -> 8500
Handling connection for 8500
Handling connection for 8500
Handling connection for 8500
Handling connection for 8500
Handling connection for 8500
Handling connection for 8500
```

Go to Browser and access the consul portal using this URL: **http://localhost:8500**



Conclusion

Deploying HashiCorp Consul on Amazon EKS provides a robust platform for managing microservices with a service mesh. Key benefits include secure service-to-service communication, centralized traffic management through Consul intentions, and integration with AWS IAM for enhanced security. By leveraging auto-scaling node groups, persistent storage, and Consul's service discovery, this setup ensures scalability, reliability, and seamless application communication across namespaces. It's an ideal foundation for modern, cloud-native workloads.

Keep Learning, Keep EKS-ing!!

Feel free to reach out to me, if you have any other queries or suggestions Stay connected on LinkedIn: Mahendran Selvakumar

Stay connected on Medium: https://devopstronaut.com/